

STEP-BY-STEP MANUAL

Mobile Game Development Project

ROCK FALL

3D Game Space Shooter

Members:

Gano Spencer Chumilang
Ver Velasco Raquepo
Jan Ellisson Sebastian

BSIT - 4A

Submitted to:

Sir Jason Nacorda

January 8, 2026

Table of Contents

Project Overview ----- 1

Tools & Technology ----- 2

Installation & Setup ----- 3

Architecture & Code Walkthrough ----- 4

Game Design & UI ----- 5

Art & Sound Pipeline ----- 6

Deployment & Release Notes ----- 7

Future Work ----- 8

Appendices ----- 9

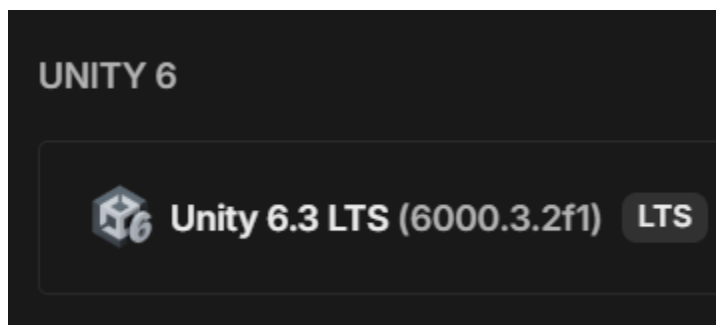
1. Project Overview

Rockfall is a 3D space survival game built in Unity. Players pilot a ship to protect a space station by evading and shooting down incoming asteroids. The goal is to survive for as long as possible, with a scoring system that rewards both longevity and defensive prowess.

2. Tools & Technology

2.1 Game Engine

- Unity Version: Unity 2025. 3 LTS
Why this matters: Unity provides strong 3D support, mobile optimization, and an easy Android build system.



2.2 Programming Language

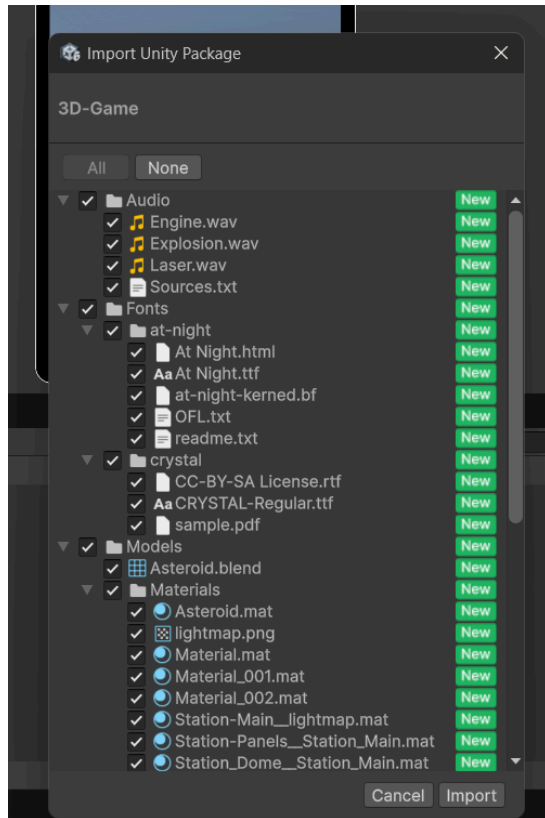
- C# – used for all gameplay scripts and logic.

2.3 Development Tools

- Unity Hub – project management
- Visual Studio – code editor
- Android SDK & JDK – required for Android builds

2.4 Assets & Resources

- 3D models: Free low-poly spaceship and asteroids
- Audio: Free sound effects for shooting and explosions



- Asset sources:
 - OpenGameArt.org

3. Installation & Setup (Step-by-Step)

3.1 Pre-requisites

1. Install Unity Hub
2. Install Unity 2023.3 LTS
 - Include Android Build Support
 - Android SDK & NDK

- OpenJDK

3.2 Opening the Project

1. Download or unzip the project folder
2. Open Unity Hub
3. Click Open Project
4. Select the RockFall project folder

3.3 Running the Game in Editor

1. Open the `MainMenu` scene
2. Click the Play button
3. Use keyboard/mouse for testing

3.4 Building APK for Android

1. Go to File → Build Settings
2. Select Android
3. Click Switch Platform
4. Click Build and Run
5. Ensure USB debugging is enabled on device

4. Architecture & Code Walkthrough

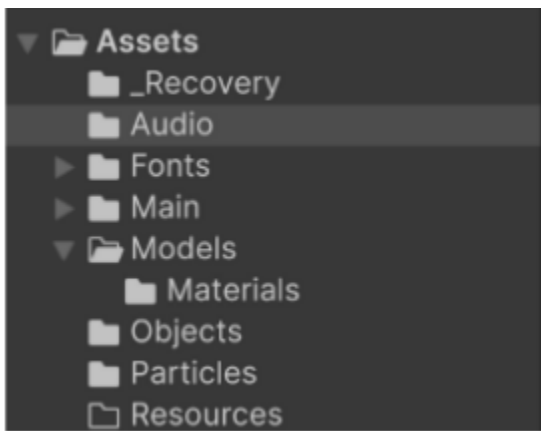
4.1 High-Level Architecture

The game follows a simple component-based architecture:

- Game Manager – controls game state

- Player Controller – movement and shooting
- Asteroid Spawner – spawns falling asteroids
- UI Manager – score and health display
- Audio Manager – sound effects

4.2 Folder Structure



4.3 Key Scripts

- GameManager.cs – handles game start, game over, restart
- PlayerController.cs – spaceship movement and firing
- AsteroidSpawner.cs – creates falling asteroids
- ScoreManager.cs – tracks player score

4.4 Data Storage

- Player score is stored using PlayerPrefs
- No cloud or online storage used

5. Game Design & UI

5.1 Core Mechanics

- Move spaceship left/right
- Tap fire button to shoot
- Destroy asteroids to gain score
- Game ends when player collides with asteroid

5.2 UI Flow

Main Menu → Gameplay → Game Over → Restart / Exit

5.3 Accessibility

- Large UI buttons
- Simple controls
- Clear contrast between objects

5.4 Asset Attribution

All assets used are free and licensed for educational use.

6. Art & Sound Pipeline

6.1 Asset Optimization

- Low-poly 3D models
- Compressed textures
- Short audio clips

6.2 Naming Conventions

- `player_ship.fbx`
- `asteroid_large.fbx`
- `laser_shot.wav`

6.3 Adding New Assets

1. Import asset into Assets folder
2. Assign material and scale
3. Attach to prefab

7. Deployment & Release Notes

7.1 Deployment

- Built as Android APK
- Ready for Play Store upload

7.2 Release Notes (v1.0)

- Basic space shooter gameplay
- Scoring system
- Game over screen
- Mobile controls

8. Future Work / Improvements

- Power-ups

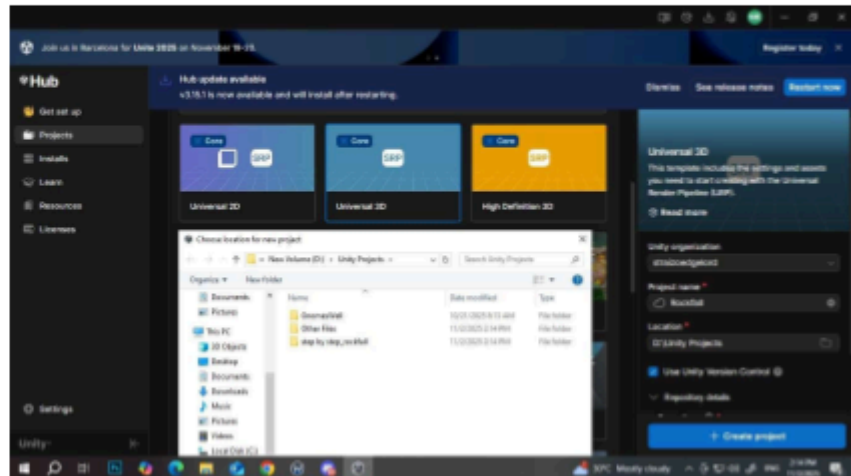
- Boss enemies
- Leaderboard
- Soundtrack music
- Difficulty selection

9. Appendices

- https://github.com/ShalamanShalaman/ShalamanShalaman-PEBSIT004_Rockfall3D_ChumilangGanoD_Raquep-oVerV_SebastianEllissonG.git

STEP BY STEP ROCKFALL

Create game
in 3D through
Unity Hub

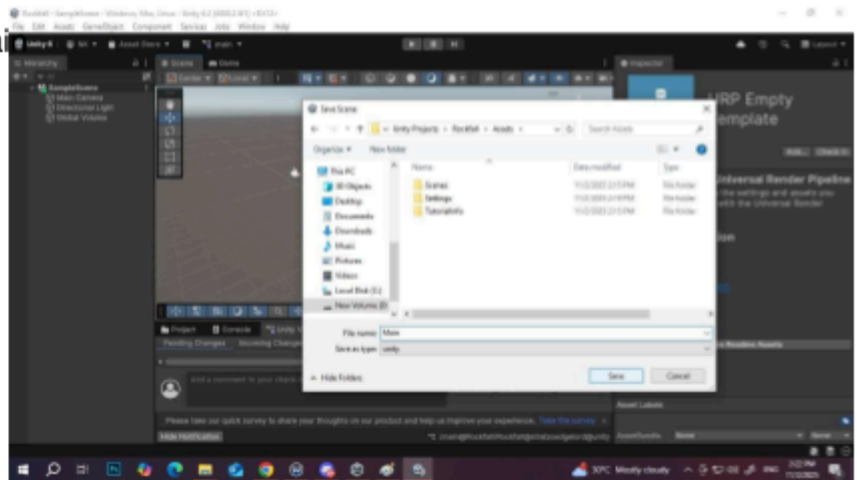


...

STEP 2

X

Save scene as Ma



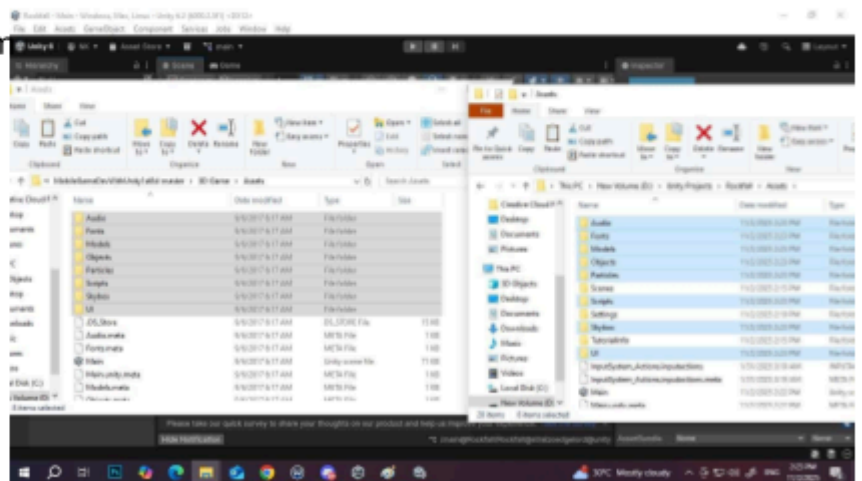
3

...

STEP 3

X

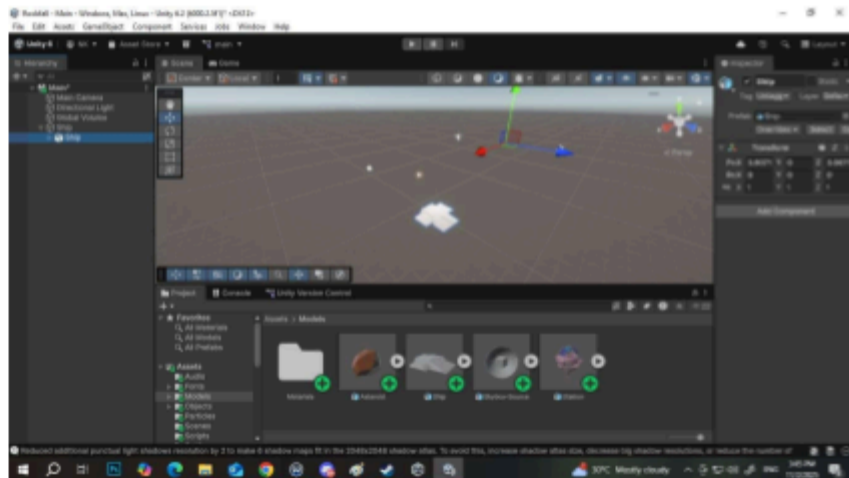
Import assets from



4

STEP 4

Make new
GameObject, rename as
Ship and drag imported
ship model to it as child
Rename ship
model as Graphics then
reset position

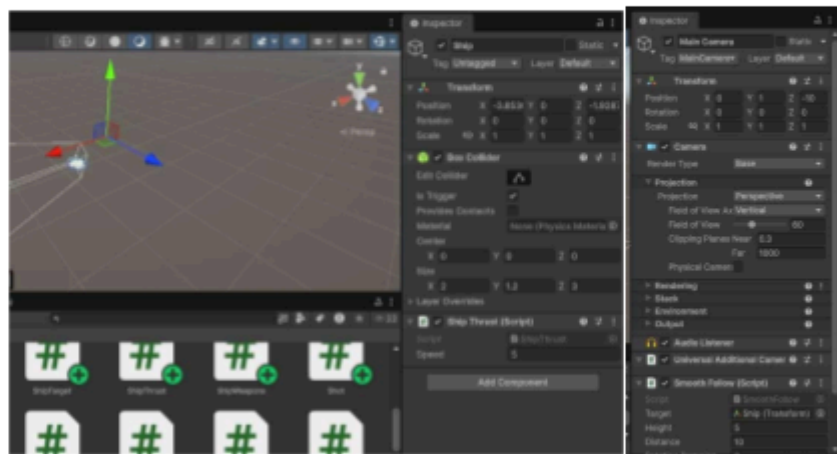


5

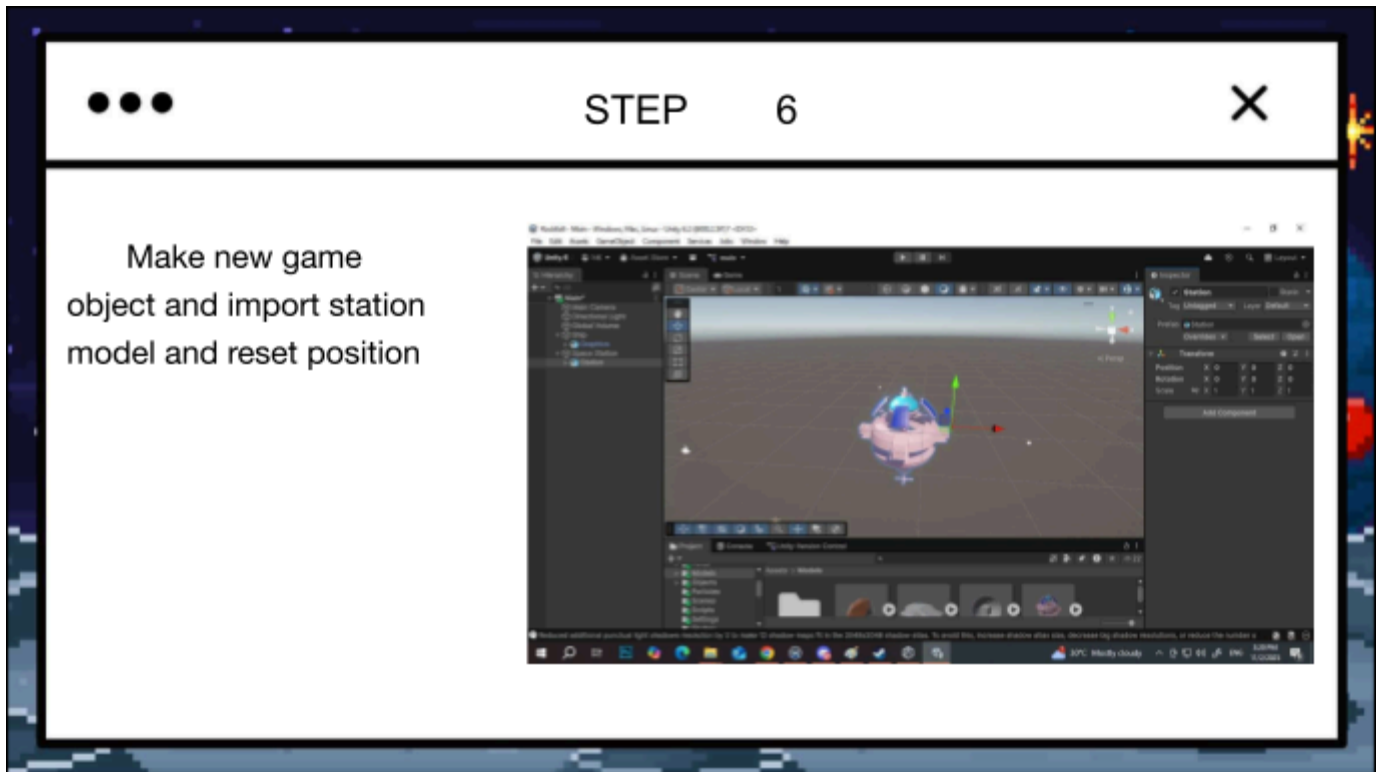
STEP 5

Add box
collider, set Is Trigger
on and size to 2, 1.2, 3
then add ship thrust
script

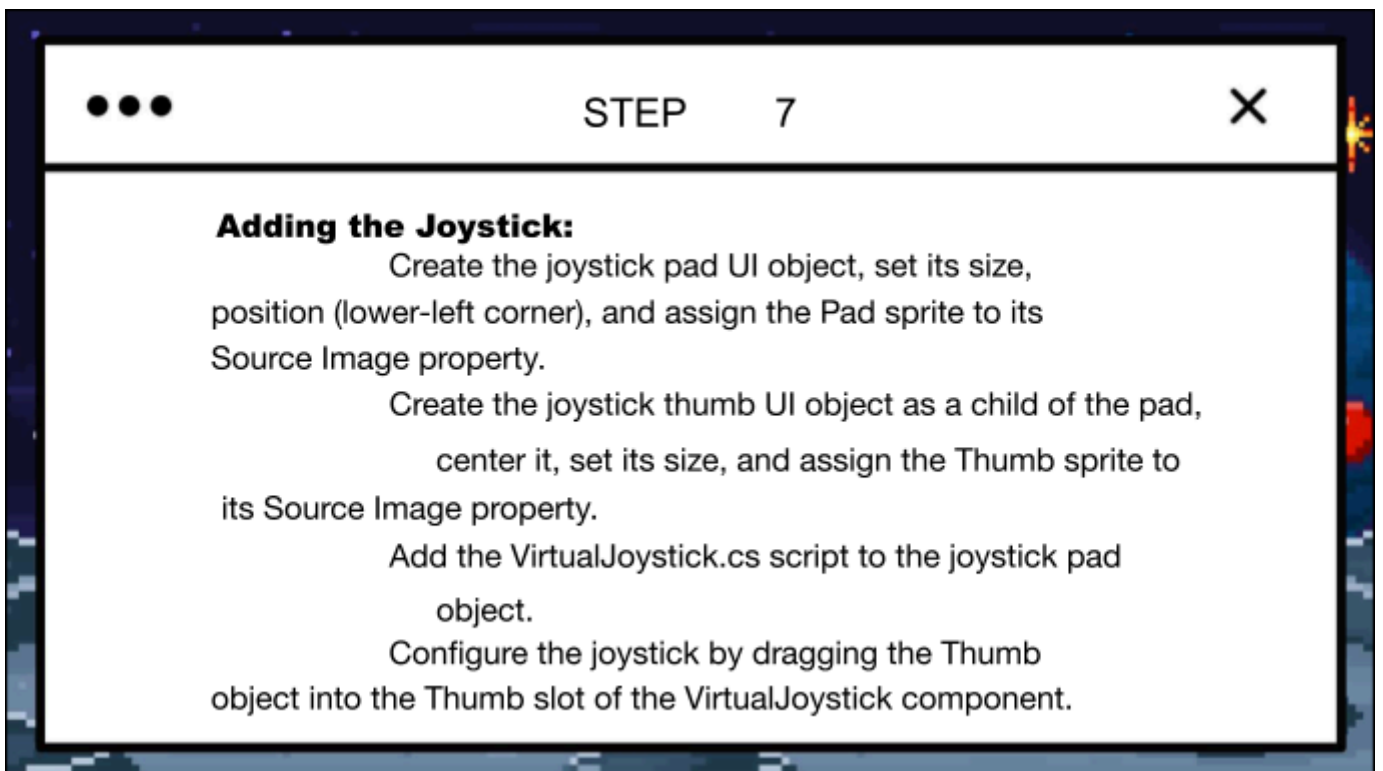
Add smooth
follow script to main
camera to follow the
ship, set target as Ship
under the script



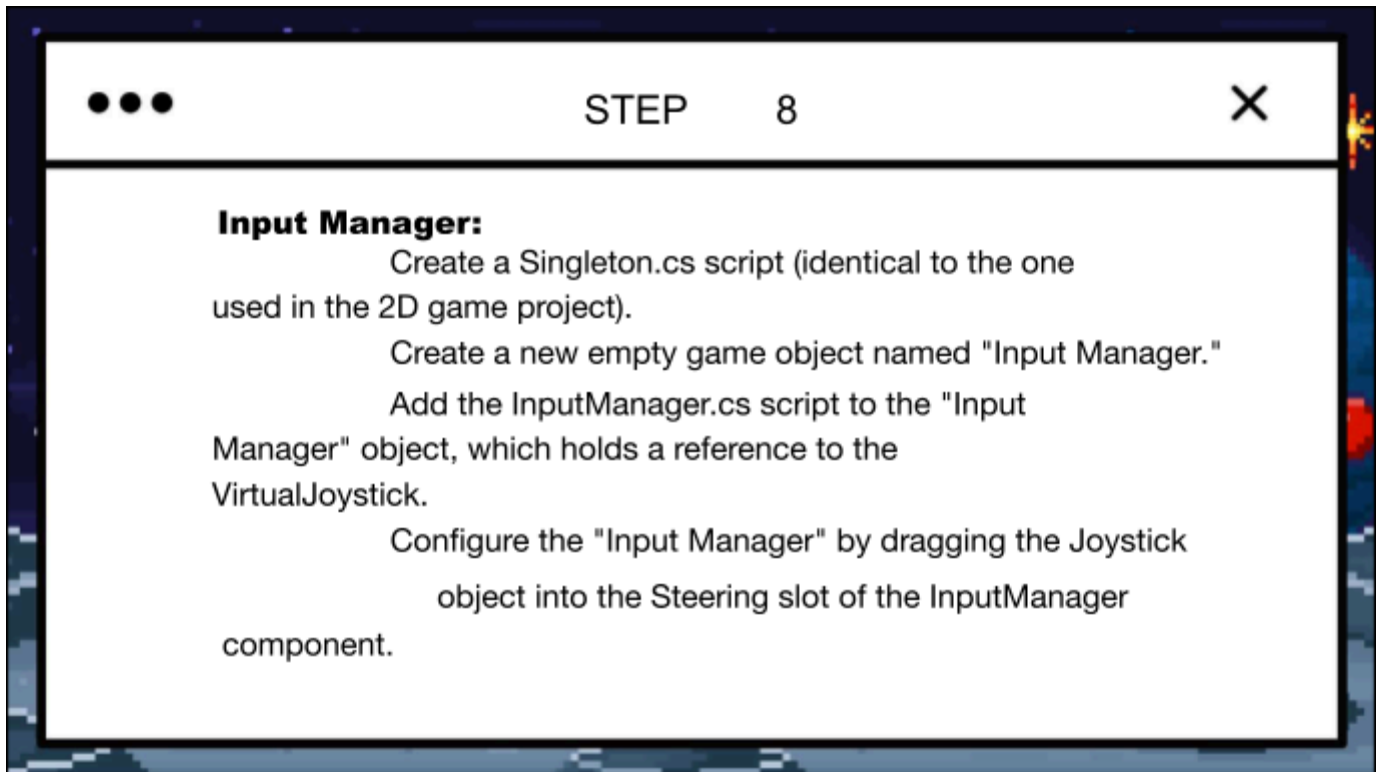
6



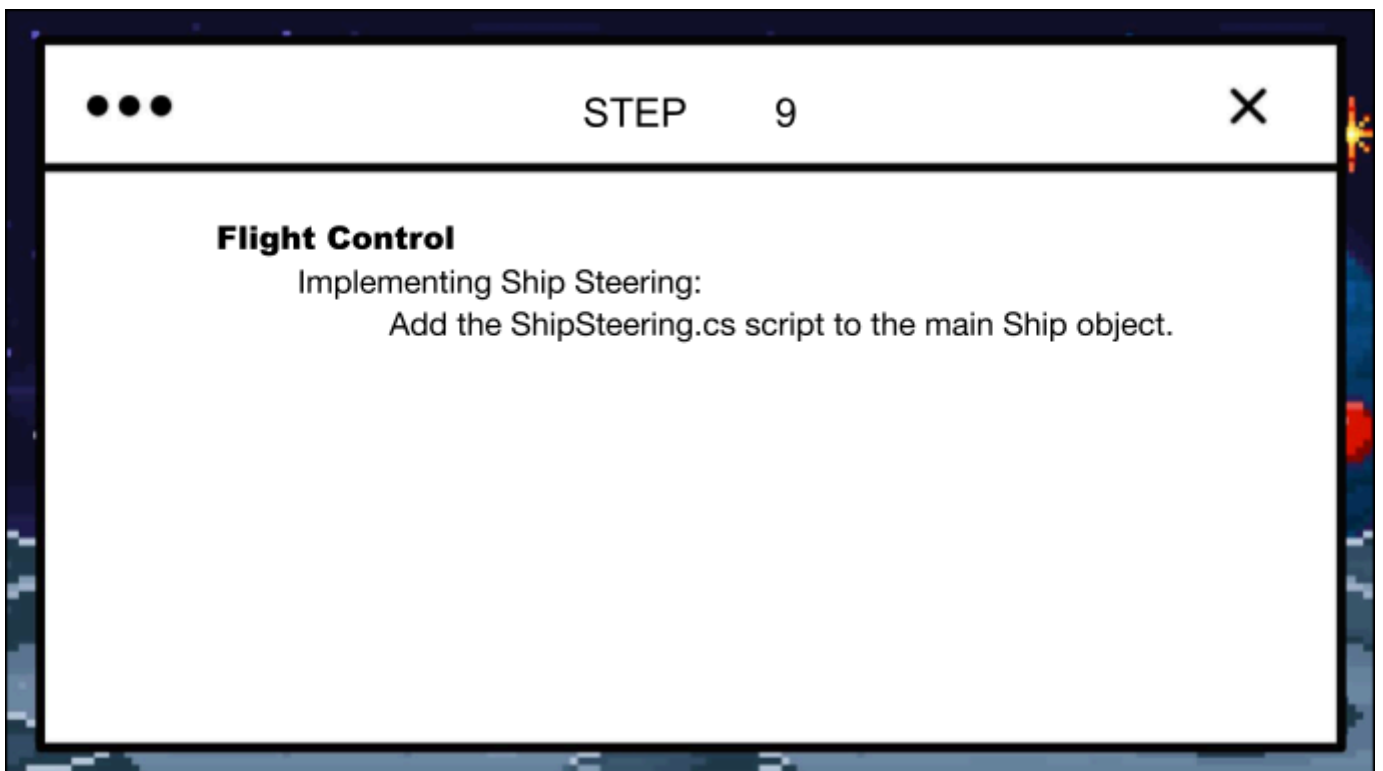
7



8



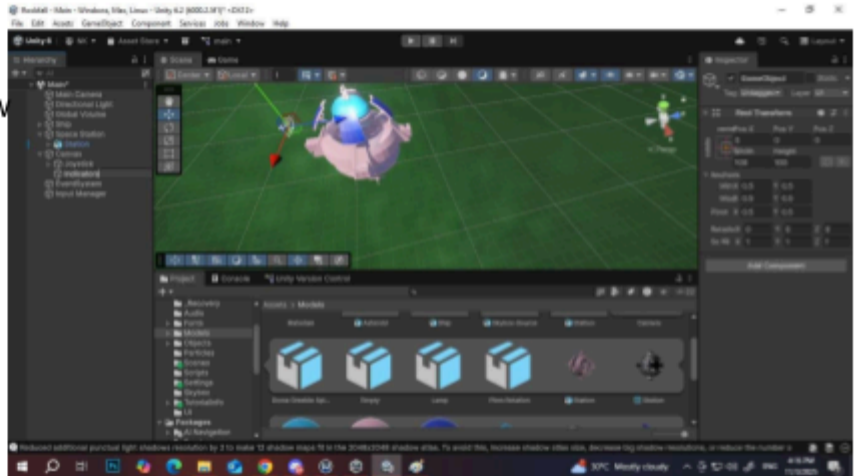
9



10

STEP 10

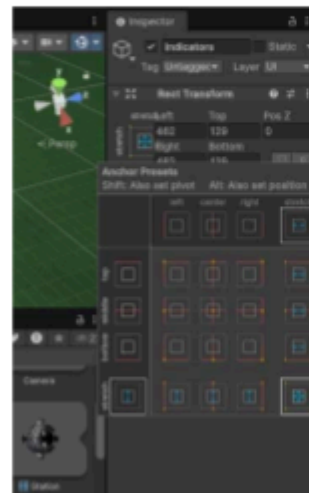
Skipped through flight controls guide, make a new child under Canvas called Indicators



11

STEP 11

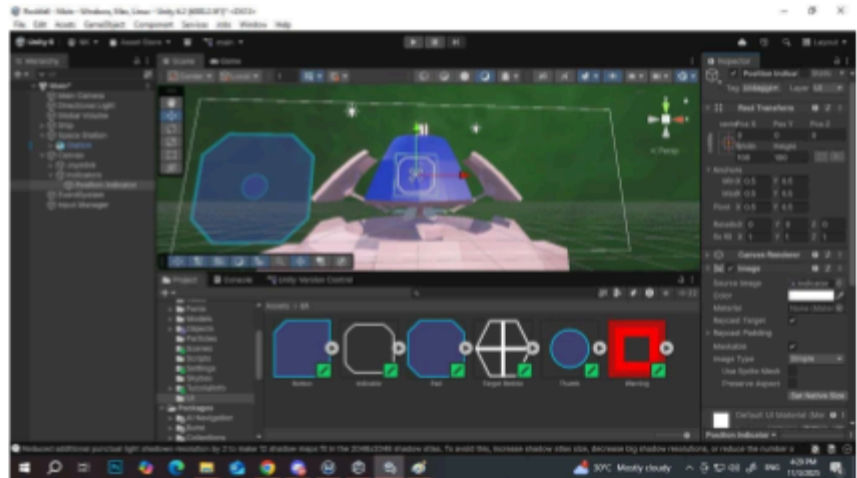
Set Indicators to stretch horizontally and vertically



12

STEP 12

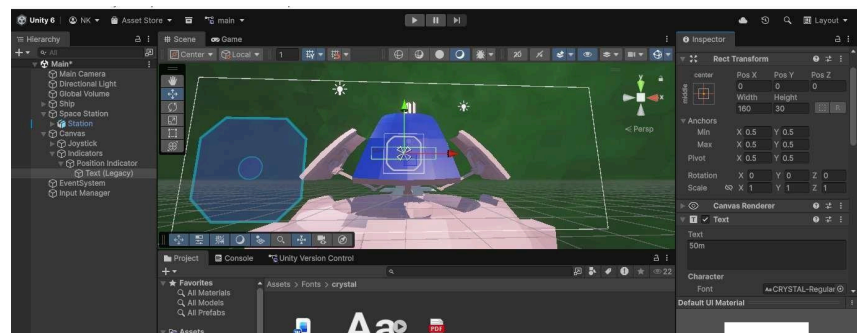
Make a new Image through GameObject-UI-Image and name it Position Indicator, make it a child of Indicators and add Indicator sprite as its source image



13

STEP 13

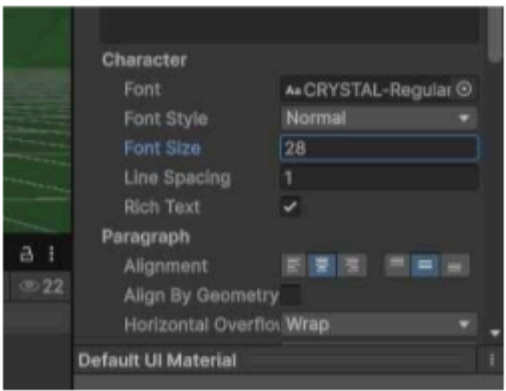
Add a Text child under Position Indicator, change its position to center middle, X and Y to 0, set text to 50m then add custom font Crystal



14

...
STEP 14
✕

Set font size to 28 and alignment to center both horizontally and vertically



15

...
STEP 15
✕

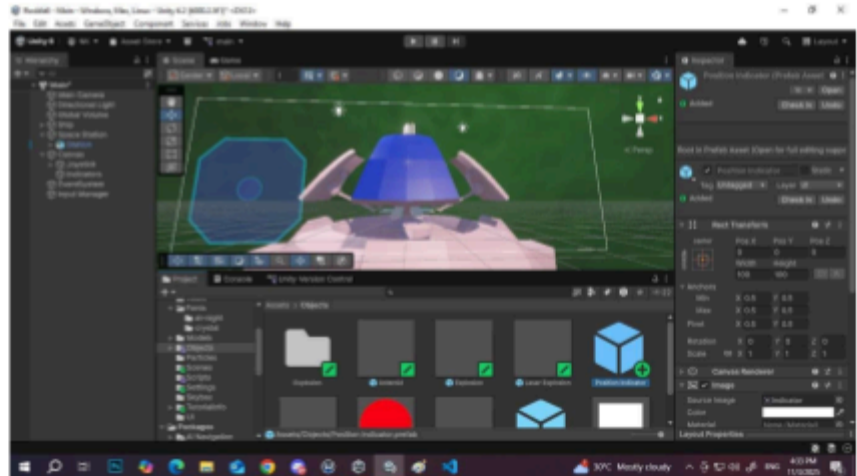
Make new script Indicator.cs then add it to position indicator and drag the text to the distance label area



16

STEP 16

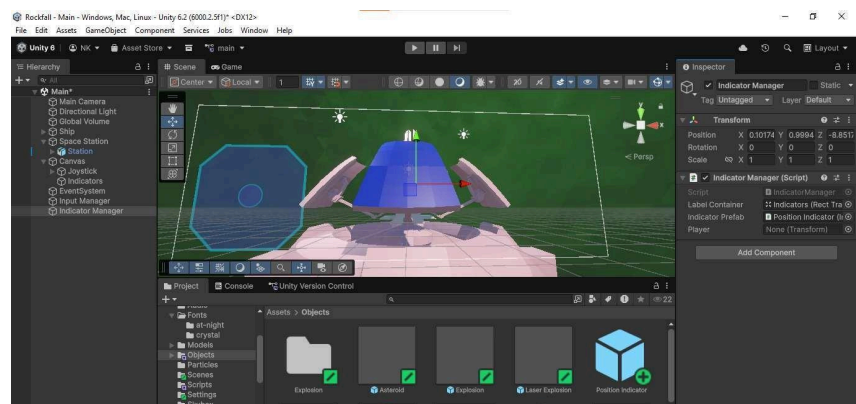
Turn the Position Indicator into a prefab and delete it from the scene sheesh



17

STEP 17

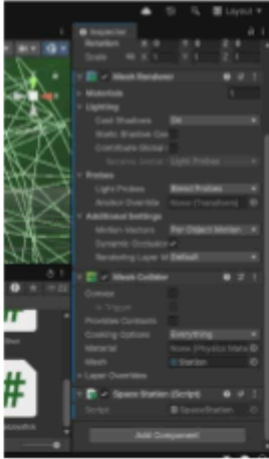
Create new game object Indicator Manager, add Indicator Manager script, drag Indicators object to label container and Position Indicator to indicator prefab



18

...
STEP 18
✕

Add Space Station script to Station



19

...
STEP 19
✕

Create a new game object named "Shot" and add a rigidbody component to the object. Then make sure Use Gravity is turned off, and Is Kinematic is turned on.



20

STEP 20

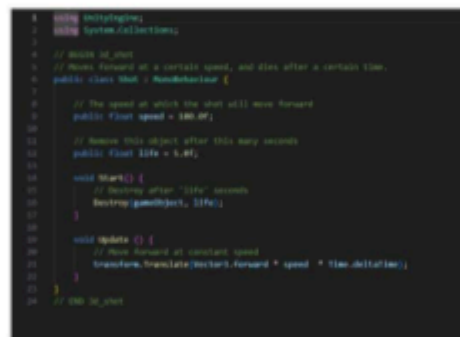
Add a sphere collider to the object. Set its radius to 0.5, and ensure that its center is (0,0,0). The Is Trigger setting should be turned on.



21

STEP 21

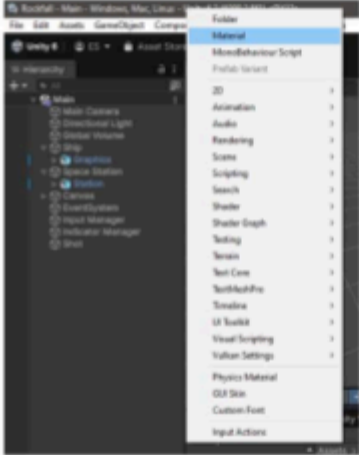
Add a new C# script called Shot to the object.



22

...
STEP 22
✕

Create a new material named Shot and set its colors.




23

...
STEP 23
✕

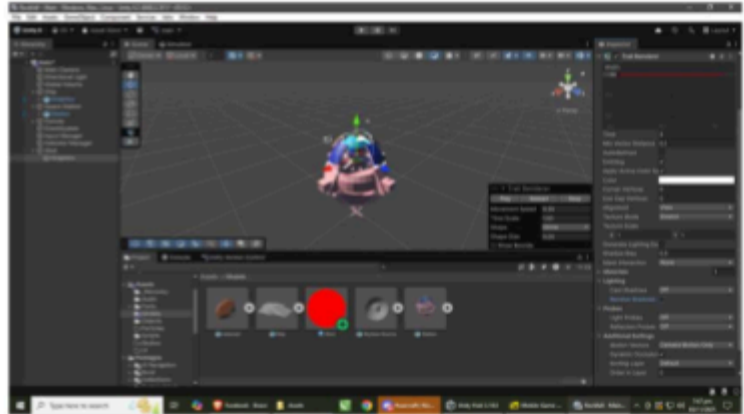
Create a new empty object, and name it "Graphics". Make it a child of the Shot object, and set its position to (0,0,0). Add a new Trail Renderer component to the Graphics object.




24

STEP 24

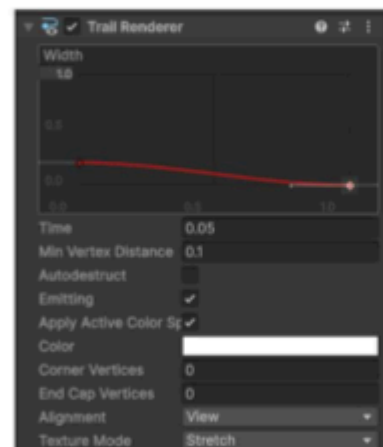
Once it's added, the Cast Shadows, Receive Shadows, and Use Light Probes should all be turned off. Next, set Time to 0.05, and Width to 0.2



25

STEP 25

Double-click in the curve view (below the Width field), and a new control point will appear. Drag this new control point to the bottom-right of the curve view.



26

...

STEP 26

X

Open the list of Materials, and drag in the Shot material that you just created.



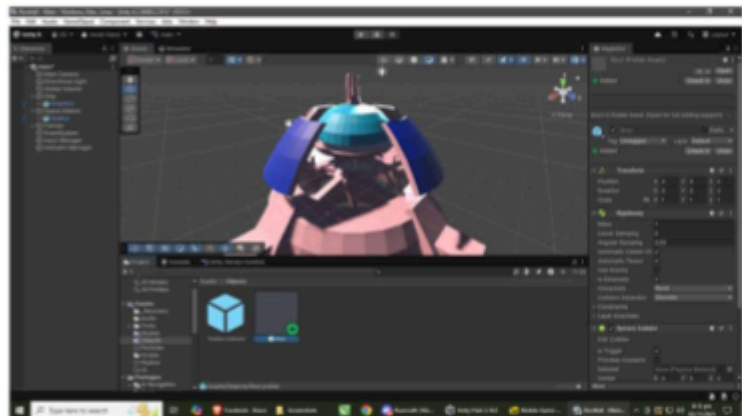
27

...

STEP 27

X

Drag the Shot object from the scene into the Objects folder. This will turn the Shot into a prefab. Delete the Shot from the scene.



28

...
STEP 28
✕

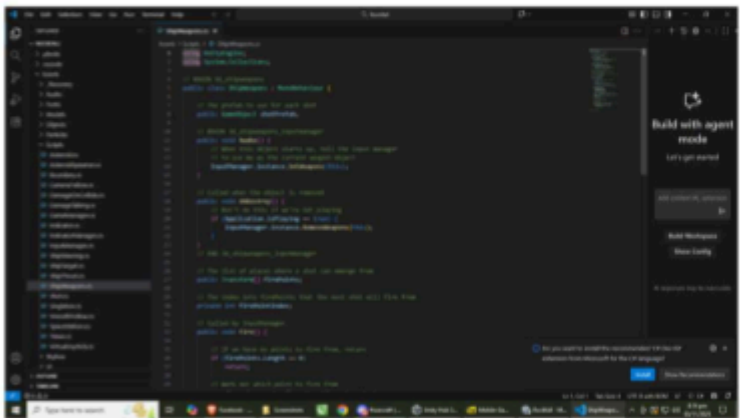
Select the Ship, add a new C# script called ShipWeapons.cs,



29

...
STEP 29
✕

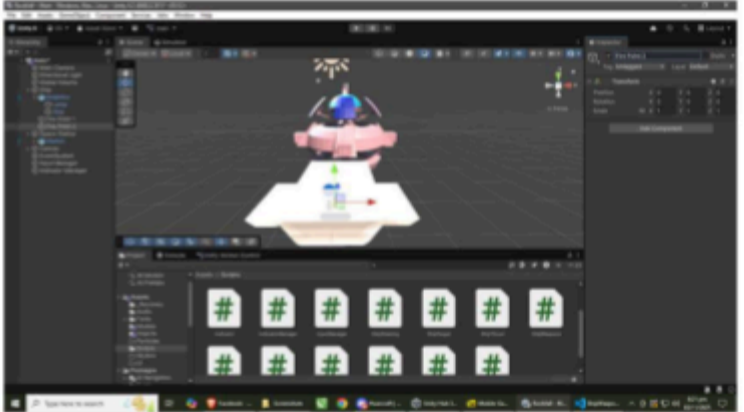
Add the following code to ShipWeapons.cs



30

● ● ●
STEP 30
✕

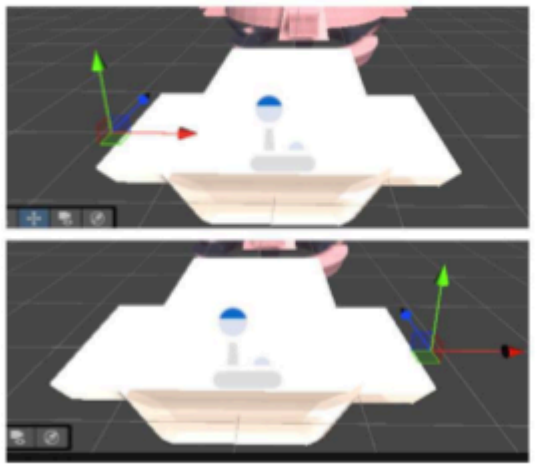
Create a new empty game object, and name it "Fire Point 1". Make it a child of the Ship object, and then duplicate it by pressing Ctrl-D



31

● ● ●
STEP 31
✕

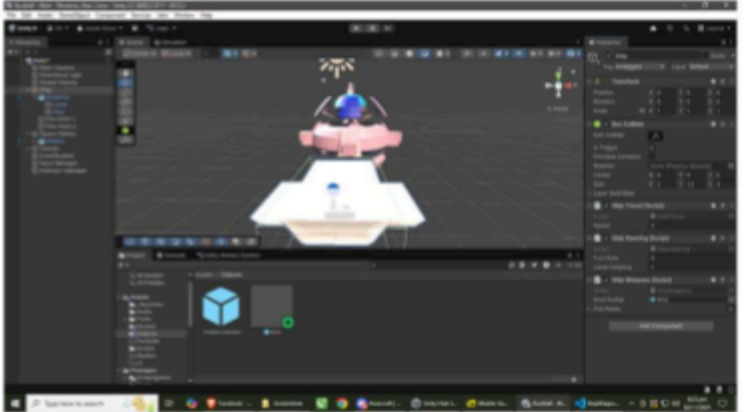
Set the position of Fire Point 1 to (-1.9, 0, 0). This will place it to the left of the ship. Set the position of Fire Point 2 to (1.9, 0, 0). This will place it to the right of the ship



32

● ● ●
STEP 32
✕

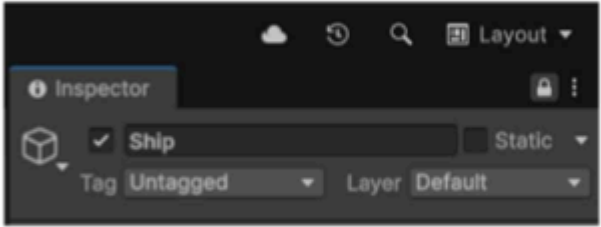
Drag the Shot prefab that you created in the earlier section to the ShipWeapons's Shot Prefab slot.



33

● ● ●
STEP 33
✕

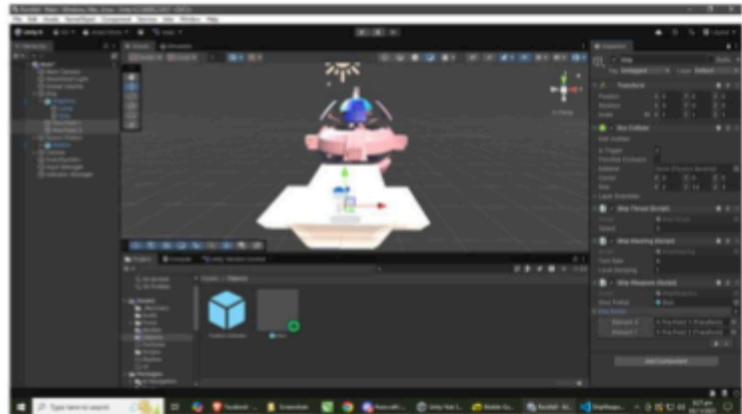
Select the Ship, and click the lock at the top right of the Inspector. This will lock the Inspector, and means that the object that the Inspector is showing won't change when you select another object.



34

STEP 34

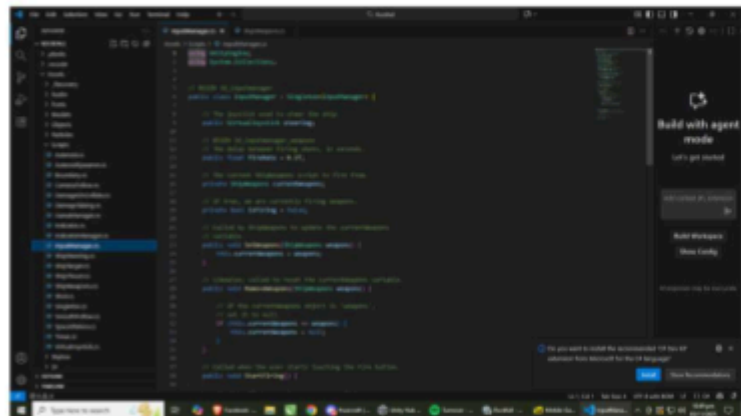
Next, select both Fire Point objects in the Hierarchy by clicking Fire Point 1 and then holding the Ctrl key, and clicking Fire Point 2. Then, drag these two objects onto the ShipWeapons' Fire Points slot. Be sure to drag it onto the text "Fire Points" (and not anything below it), or it won't work. After that unlock the inspector.



35

STEP 35

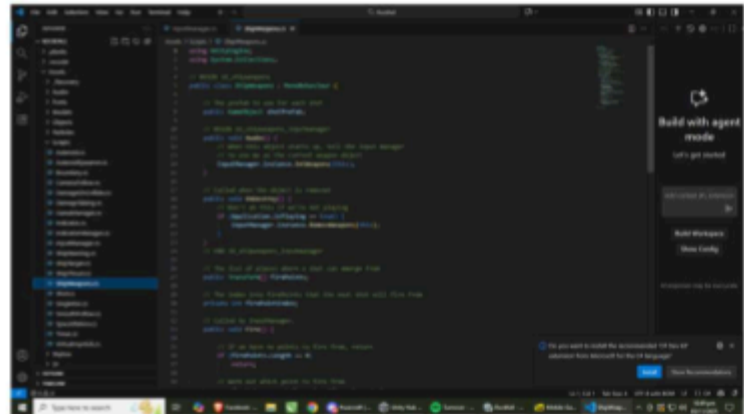
Add the ShipWeapons management code to InputManager, by adding the following properties and methods to the Input Manager class.



36

STEP 36

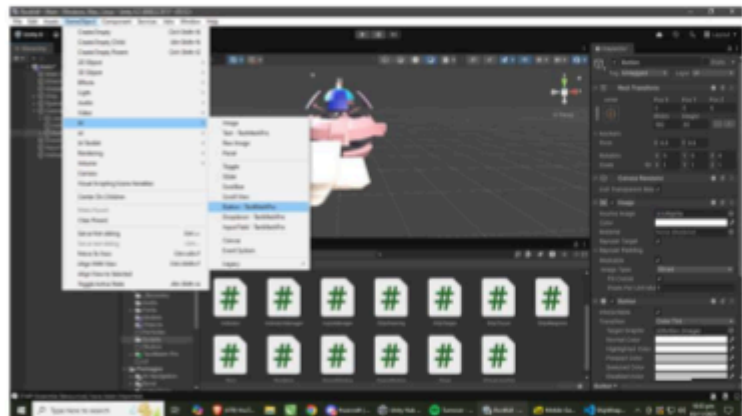
We then need to add the InputManager-communicating code to ShipWeapons by adding the following methods to the ShipWeapons class.



37

STEP 37

Create a new button by opening the GameObject menu, and choosing UI → Button. Name the new button “Fire Button”.



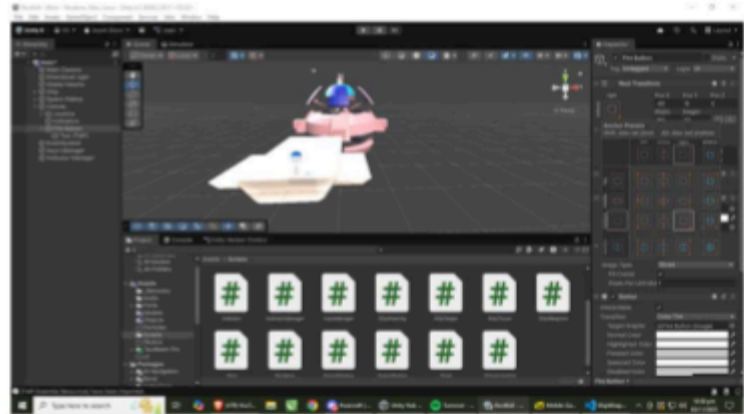
38



STEP 38



Set both the anchors and the pivot of the button to Bottom Right by clicking on the Anchor button at the top-left of the Inspector, holding the Alt key, and clicking on the Bottom Right option.



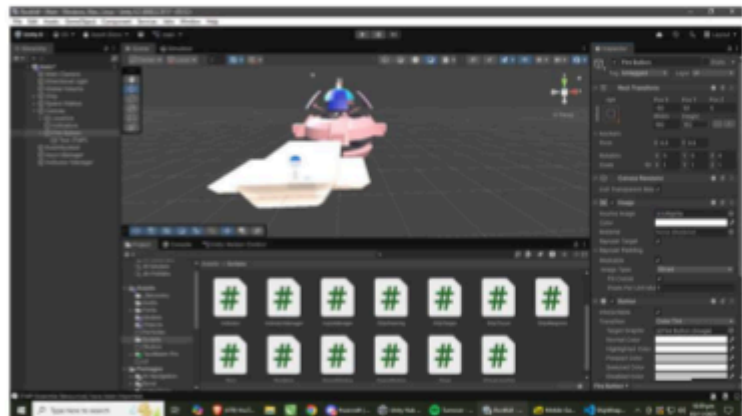
39



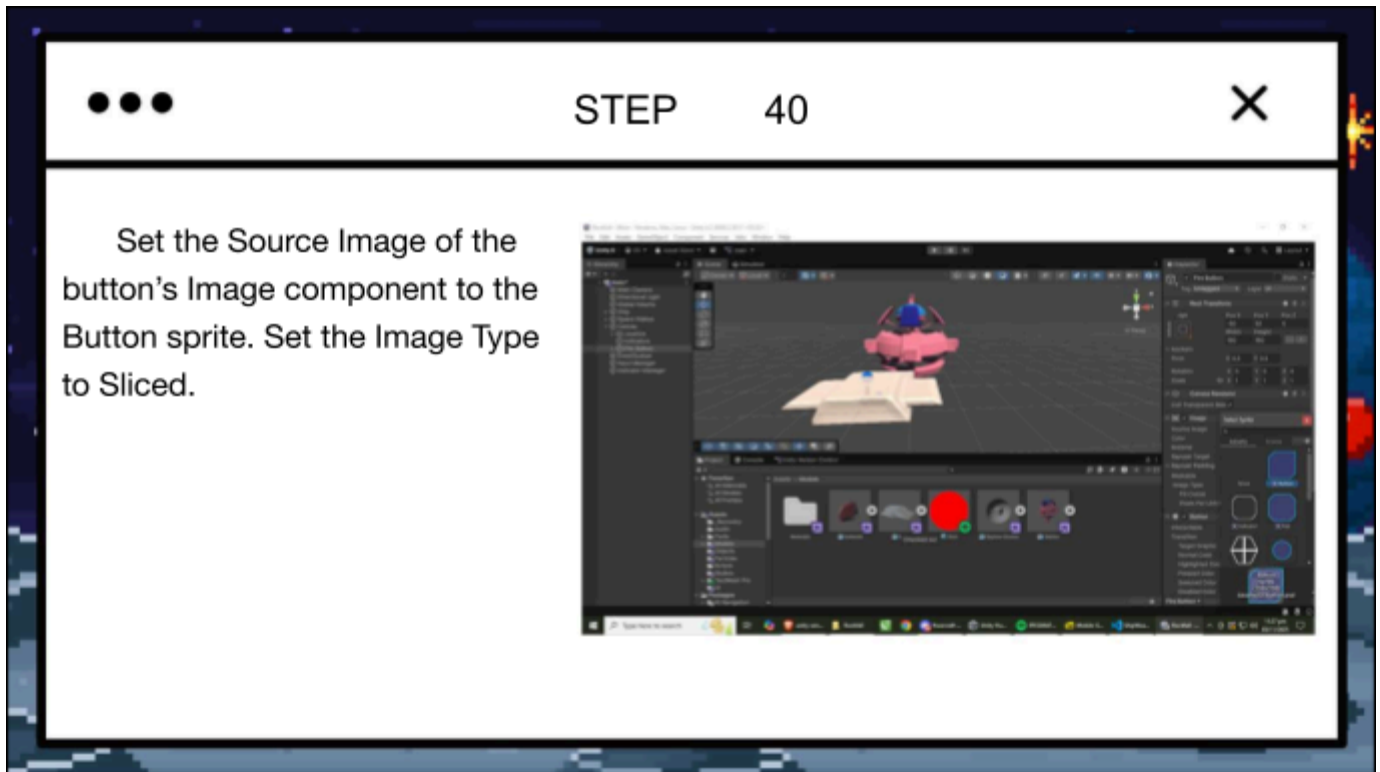
STEP 39



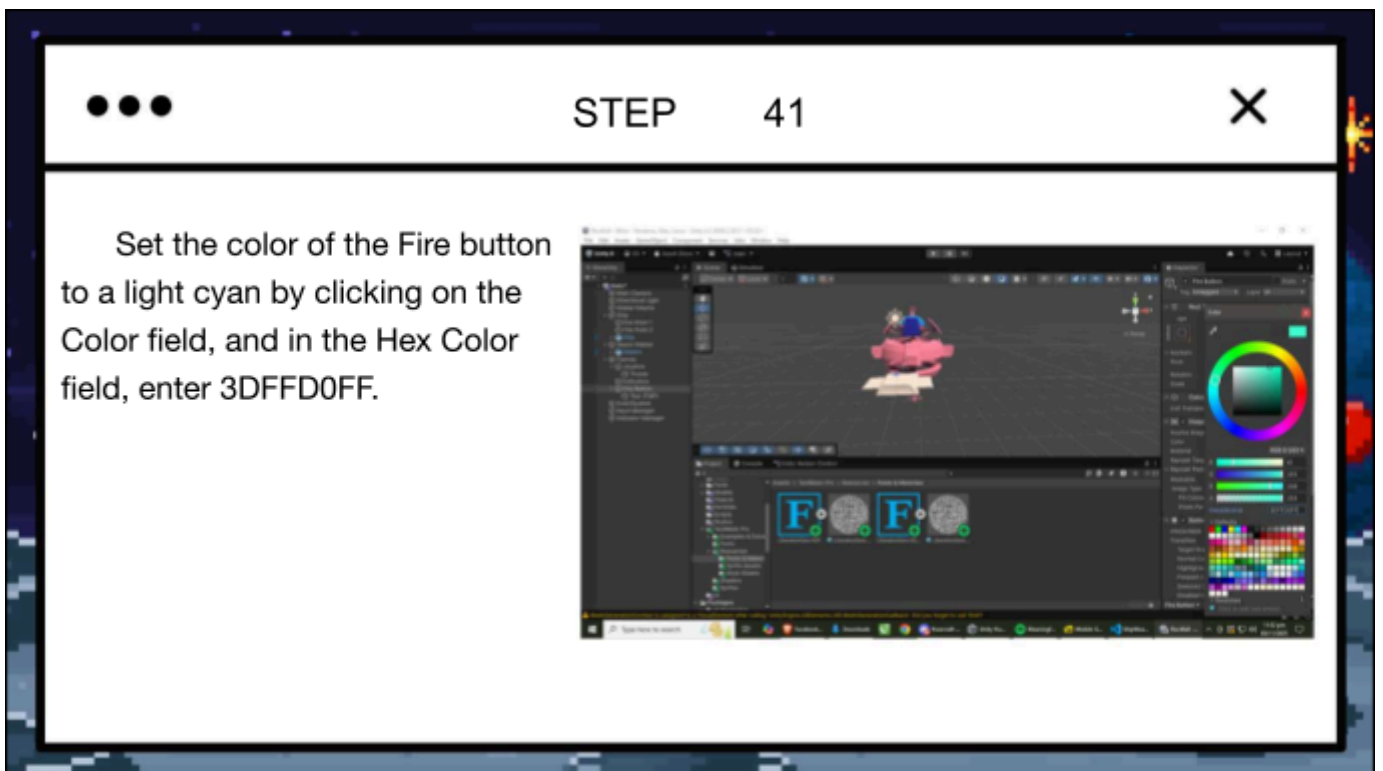
Next, set the position of the button to $(-50, 50, 0)$. This will place the button at the bottom-right of the canvas. Set both the width and height of the button to 160.



40



41



42

...
STEP 42
✕

Select the Fire Button object, and click on the settings icon at the top right of the Button component. Click Remove Component.

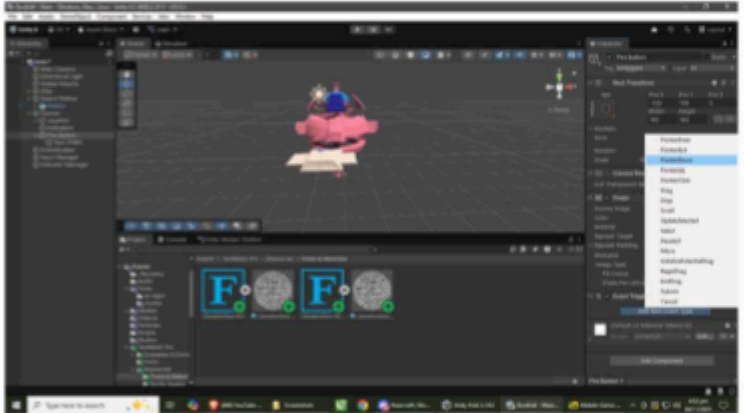


The screenshot shows the Unity Inspector window with a 'Fire Button' object selected. The 'Button' component is highlighted in the 'Components' list on the right. A context menu is open, showing the 'Remove Component' option at the top.

43

...
STEP 43
✕

Add a new Event Trigger component, and then click Add Event Type. Choose "PointerDown" from the menu that appears.



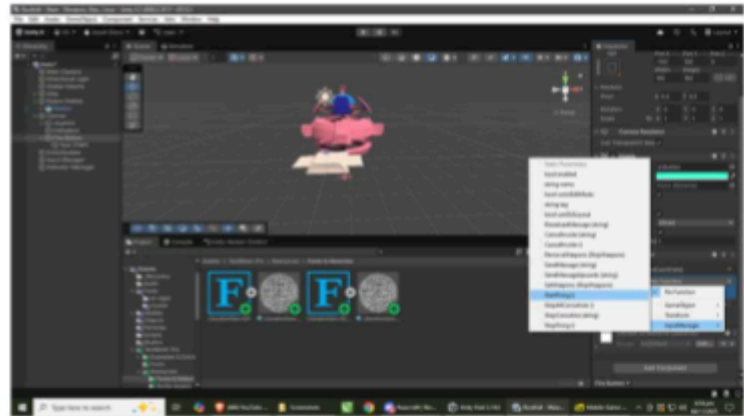
The screenshot shows the Unity Inspector window with the 'Fire Button' object selected. The 'Event Trigger' component is highlighted in the 'Components' list on the right. A context menu is open, showing the 'Add Event Type' option at the top.

44

STEP 44

Click the + button at the bottom of the PointerDown list, and a new item will appear in the list. Drag and drop the Input Manager object from the Hierarchy panel into the slot.

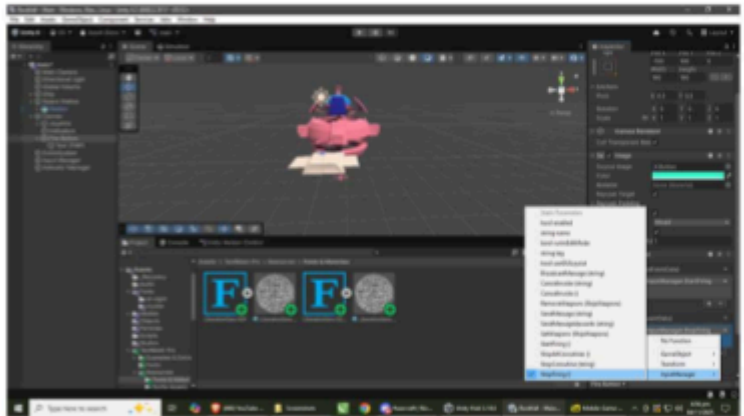
Next, change the method from “No Function” to “InputManager→StartFiring”.



45

STEP 45

Next, you need to add an event for when the finger lifts up from the screen. Click the Add Event Type again, and choose “PointerUp”. Configure this event in the same way as the PointerDown, but make the method called on the InputManager be “StopFiring”.



46

STEP 46

Name this object "Target", and make it a child of the Ship.

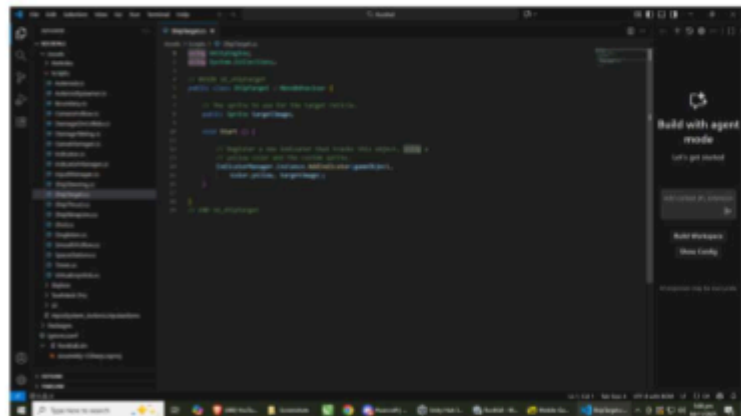
Position the Target. Set the position of the Target object to (0,0,100). This will place the target some distance away from the ship.



47

STEP 47

Add a new C# script to the Target object named ShipTarget.cs, and add the following code to it.



48

...
STEP 48
✕

Drag the "Target Reticule" sprite into the Target Image slot of the ShipTarget script.



49

...
STEP 49
✕

Make a new game object named "Asteroid". Add the asteroid model to it. Locate the Asteroid model in the Models folder.

Drag it onto the Asteroid object you just created, and rename the new child object "Graphics". Reset the Position of the Graphics object's Transform component, so that it's positioned at (0,0,0).

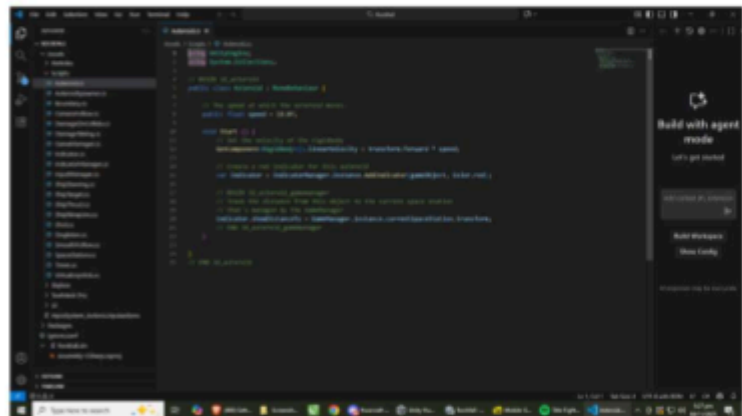


50

Add a rigidbody and sphere collider to the Asteroid object. Don't add it to the Graphics object. Once they're added, turn gravity off on the rigidbody, and make the radius of the sphere collider be 2.



Add a new C# script to the Asteroid game object, called Asteroid.cs, and add the following code to it.

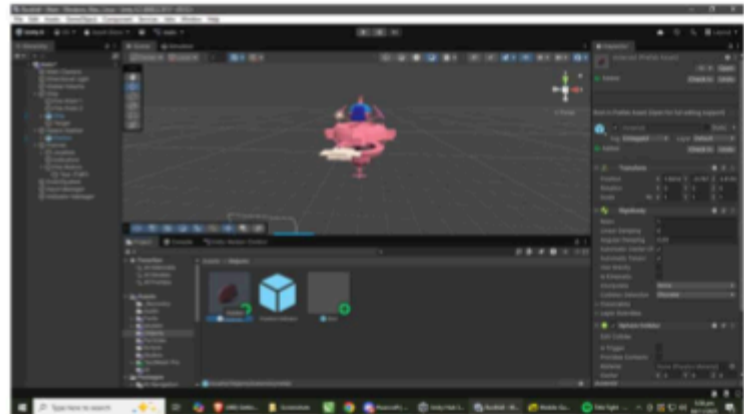




STEP 52



Drag the Asteroid object from the Hierarchy panel into the Project panel. This will create a prefab from the object. Next, delete the Asteroid from the scene.



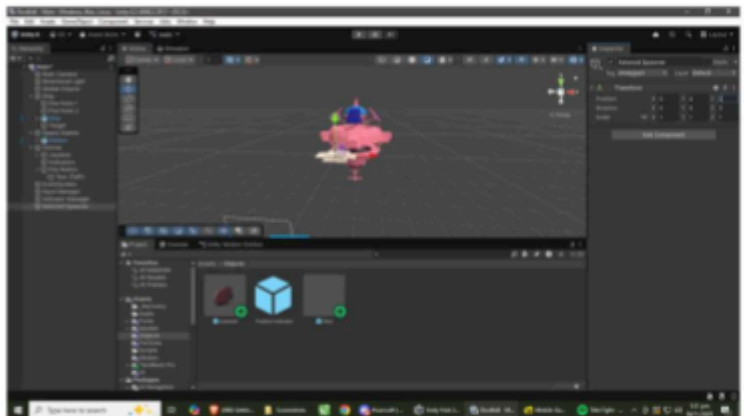
53



STEP 53



Make a new empty game object, and name it "Asteroid Spawner". Set its position to (0,0,0).



54

```

// main.ts
import { Person } from './person';
import { PersonService } from './person-service';

// 定义 Person 接口
interface Person {
  id: string;
  name: string;
  age: number;
  email: string;
}

// 定义 Person 类
class Person {
  id: string;
  name: string;
  age: number;
  email: string;
}

// 定义 PersonService 类
class PersonService {
  persons: Person[];

  constructor() {
    this.persons = [];
  }

  // 获取所有 Person 对象
  getPersons(): Person[] {
    return this.persons;
  }

  // 获取指定 id 的 Person 对象
  getPerson(id: string): Person {
    return this.persons.find(person => person.id === id);
  }

  // 创建 Person 对象
  createPerson(name: string, age: number, email: string): Person {
    const person = new Person();
    person.id = `${Date.now()}`;
    person.name = name;
    person.age = age;
    person.email = email;
    this.persons.push(person);
    return person;
  }

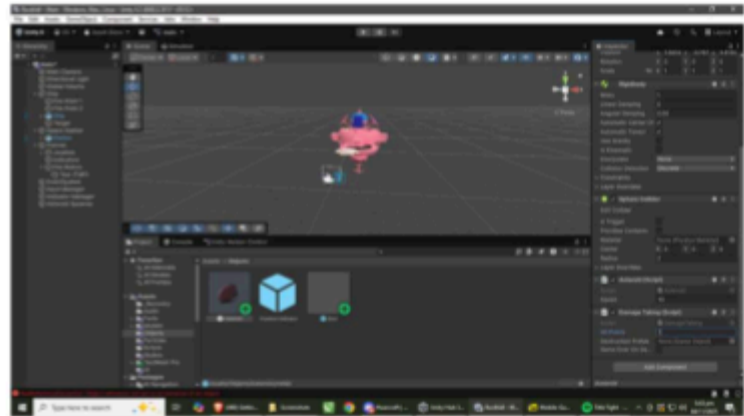
  // 更新 Person 对象
  updatePerson(id: string, name: string, age: number, email: string): Person {
    const person = this.getPerson(id);
    if (person) {
      person.name = name;
      person.age = age;
      person.email = email;
    }
    return person;
  }
}

// 测试
const personService = new PersonService();
const person = personService.createPerson('张三', 18, '1234567890@qq.com');
console.log(person);
const persons = personService.getPersons();
console.log(persons);
const person2 = personService.getPerson(person.id);
console.log(person2);
const person3 = personService.updatePerson(person.id, '李四', 20, '1234567890@qq.com');
console.log(person3);

```


STEP 58

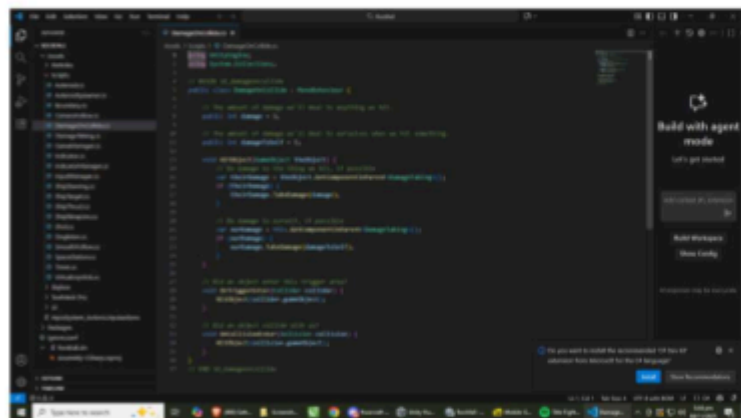
Change the asteroid's Hit Points variable to 1. This will make the asteroid very easy to destroy



59

STEP 59

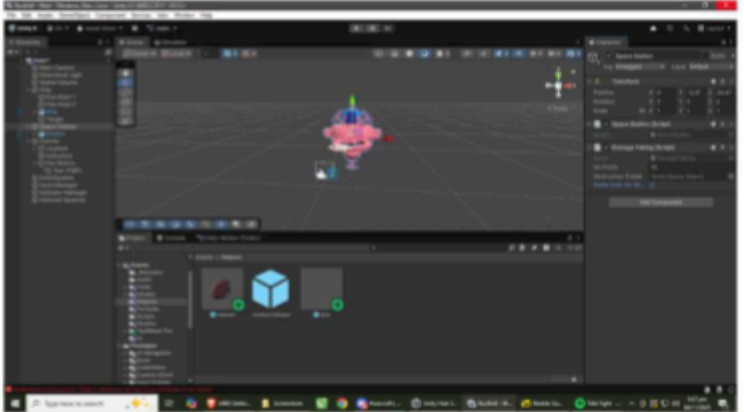
Select the Shot prefab, add a new C# script called DamageOnCollide.cs to it, and add the following code to the file.



60

...
STEP 60
✕

Select the Space Station, and add a DamageTaking script component. Turn on Game Over On Destruction.



61

...
STEP 61
✕

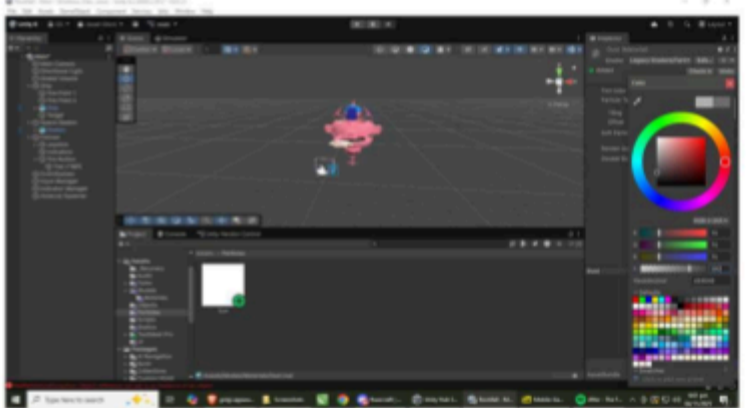
Open the Asset menu, and choose Create → Material. Name the new material "Dust".



62

...
STEP 62
✕

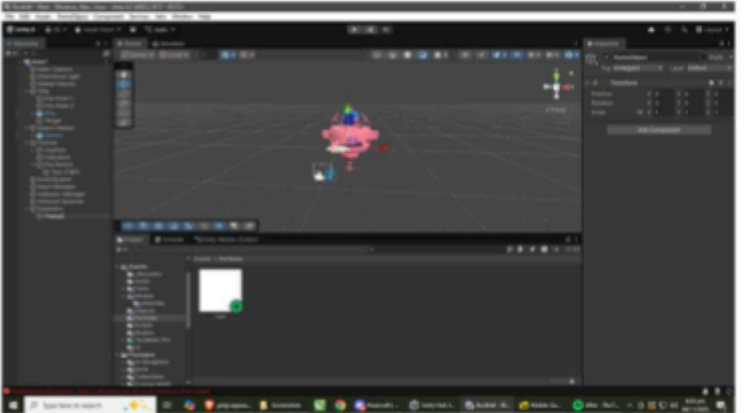
Select the material and change its shader to Particles/Additive. Next, drag the Dust texture into the Particle Texture slot. Set the tint color to a semiopaque dark gray by clicking on the Tint Color slot and selecting a color. If you'd prefer to enter specific values, enter these: (70, 70, 70, 190)



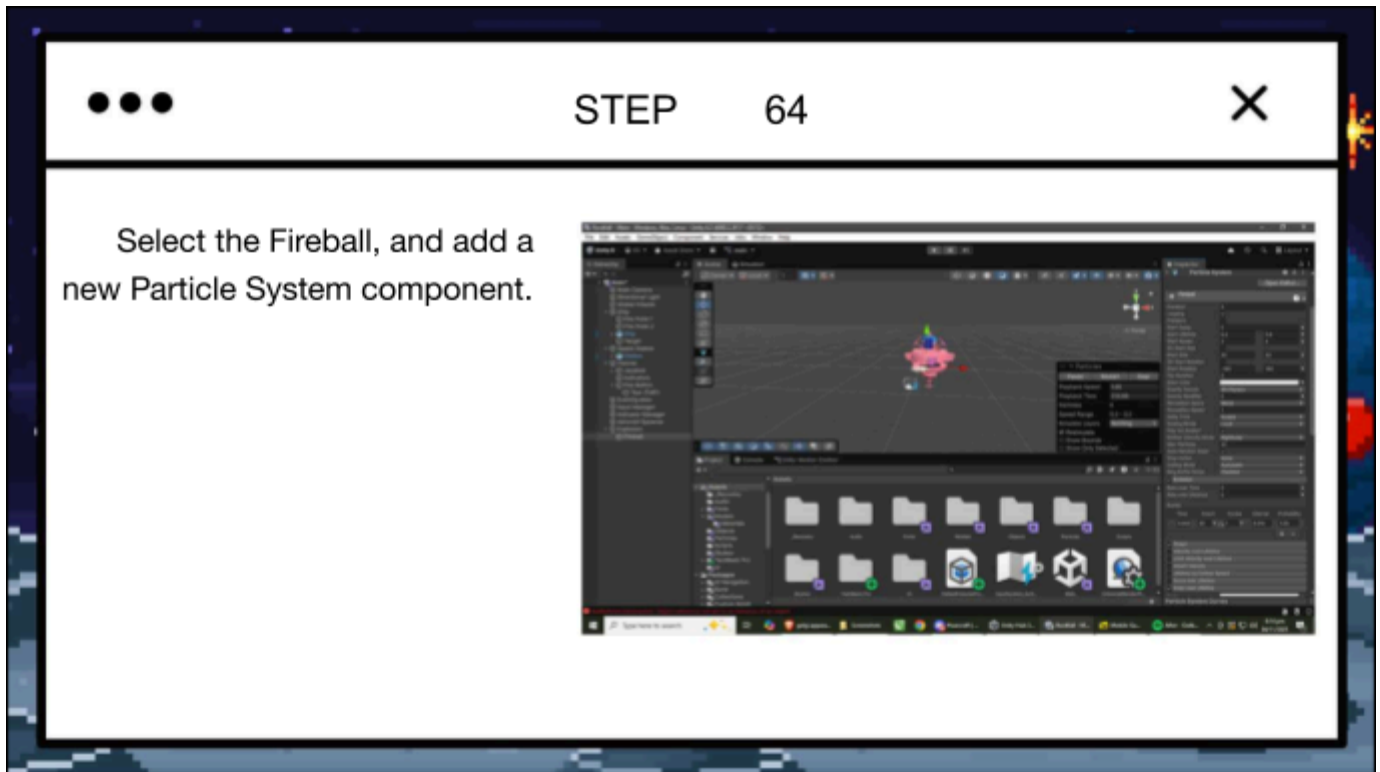
63

...
STEP 63
✕

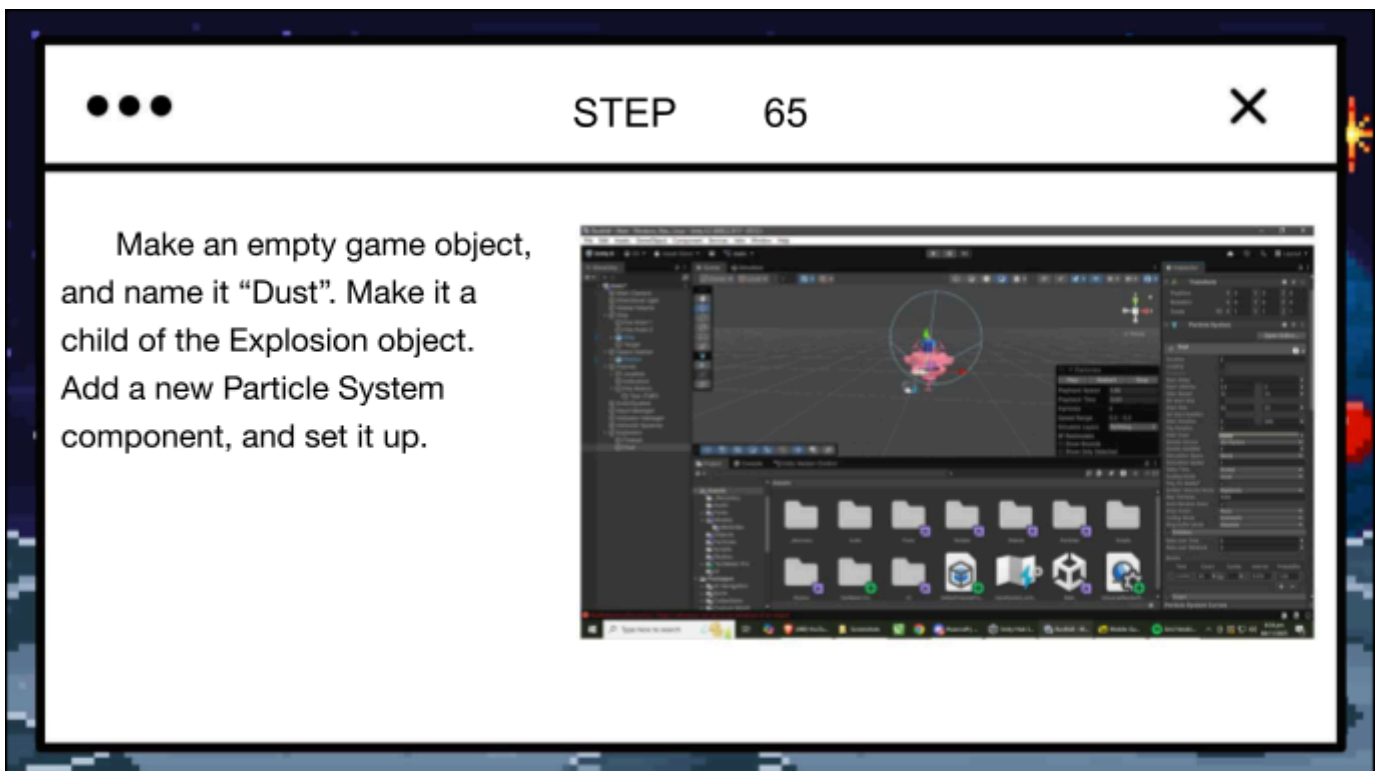
Create a new empty object, and name it "Explosion". Create a second empty object, and name it "Fireball". Make this object a child of the Explosion object.



64



65



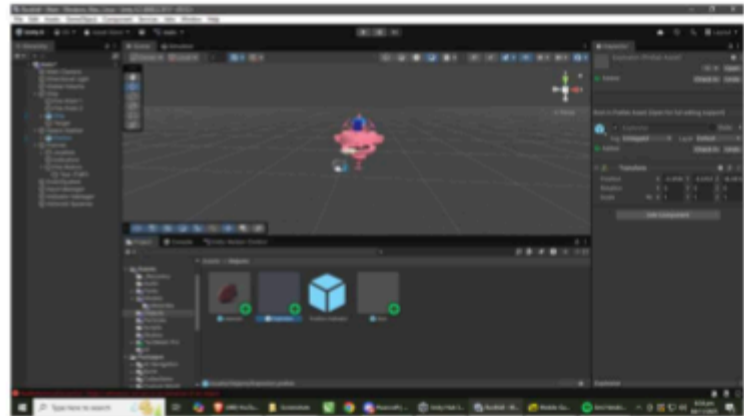
66



STEP 66



Drag the Explosion object into the Project pane, and then remove it from the scene.



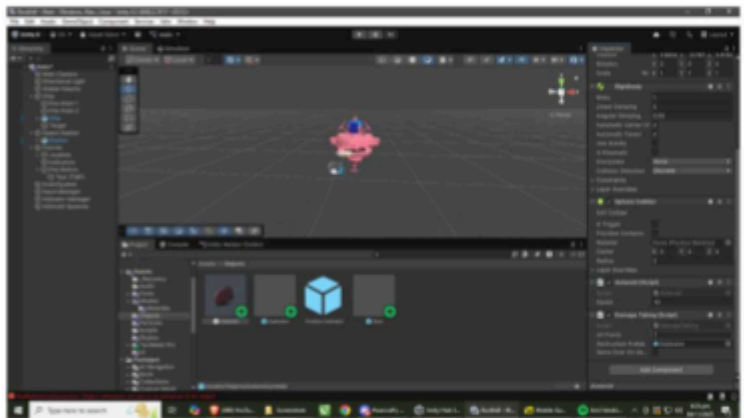
67



STEP 67



Select the Asteroid prefab, and drag the Explosion into the Destruction Prefab slot.



68

STEP 68

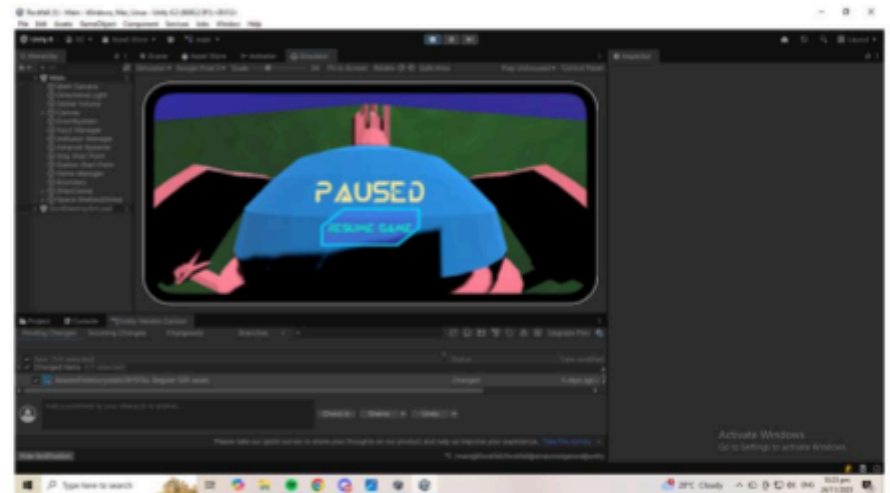
Add a Main Menu
before the game starts



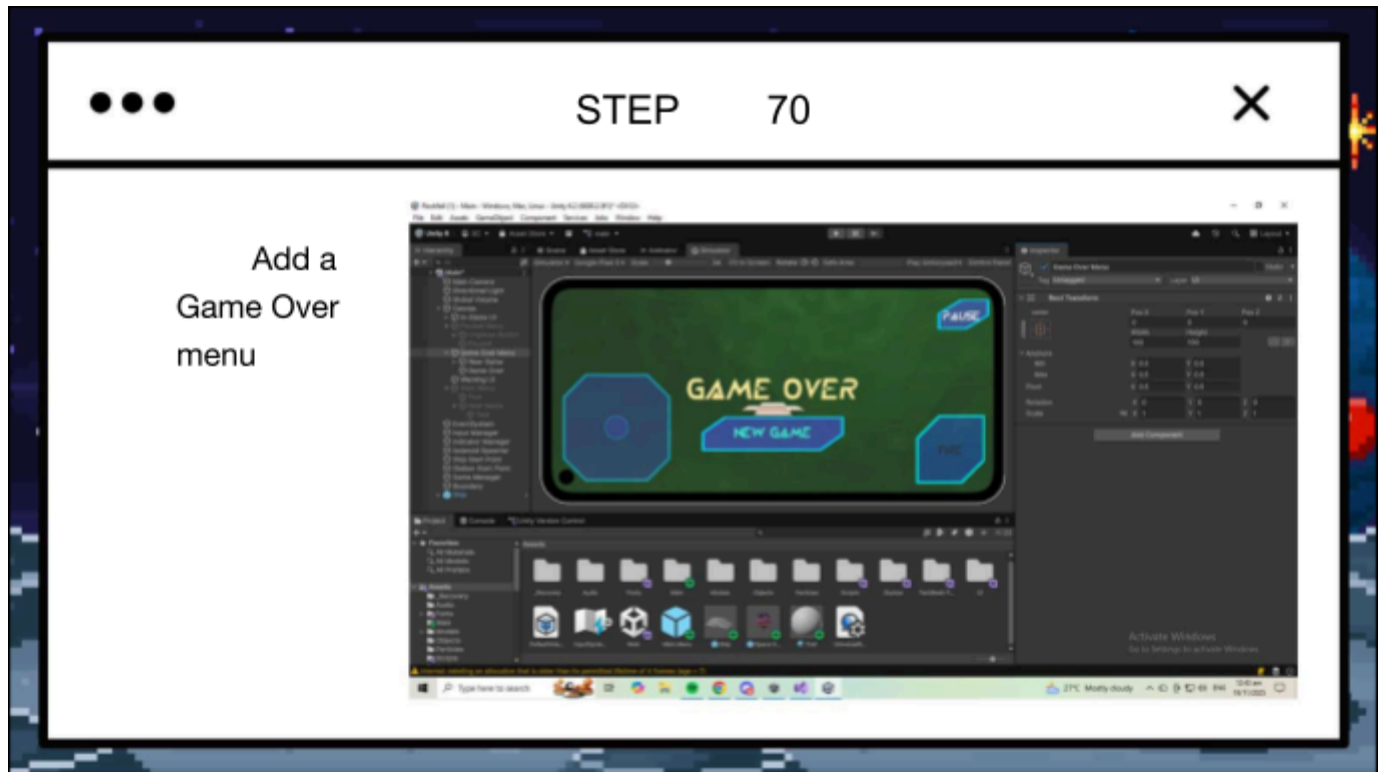
69

STEP 69

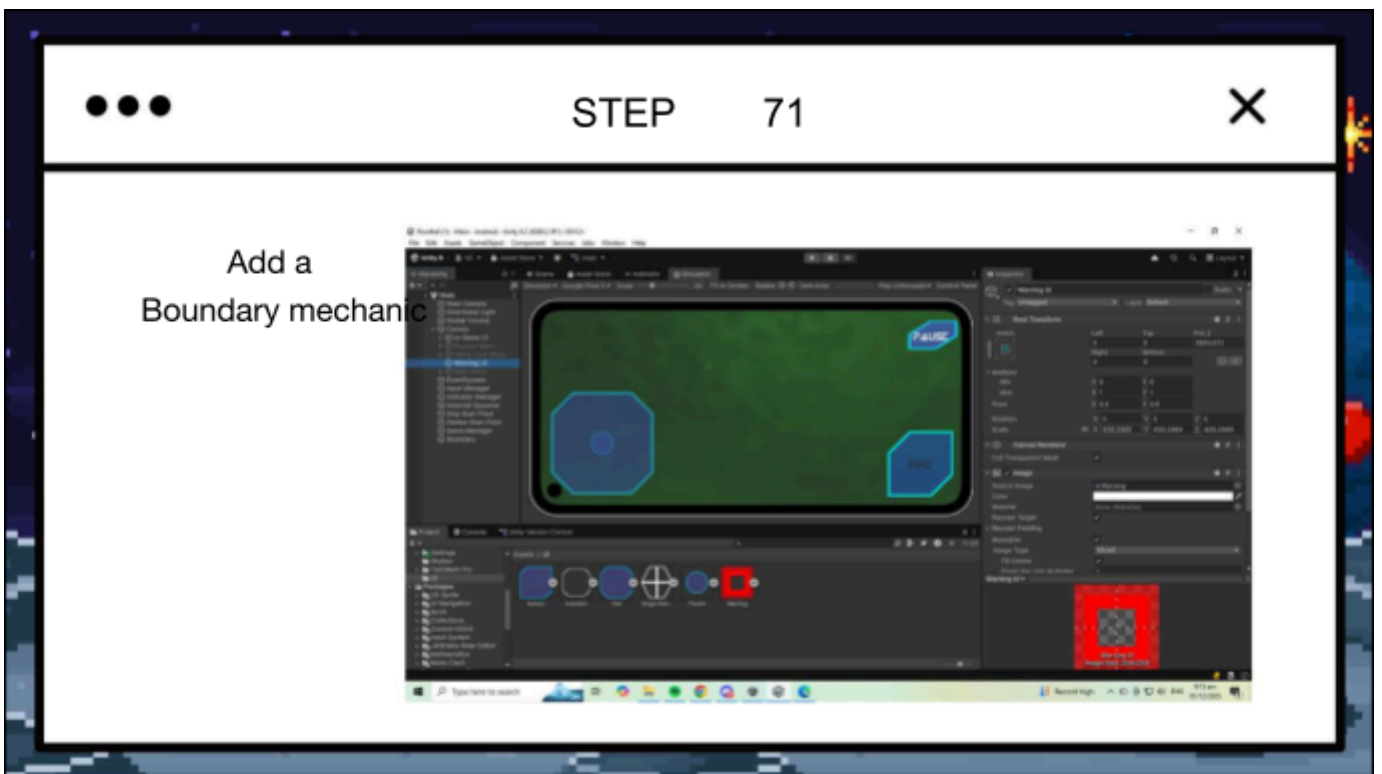
Add a Pause menu



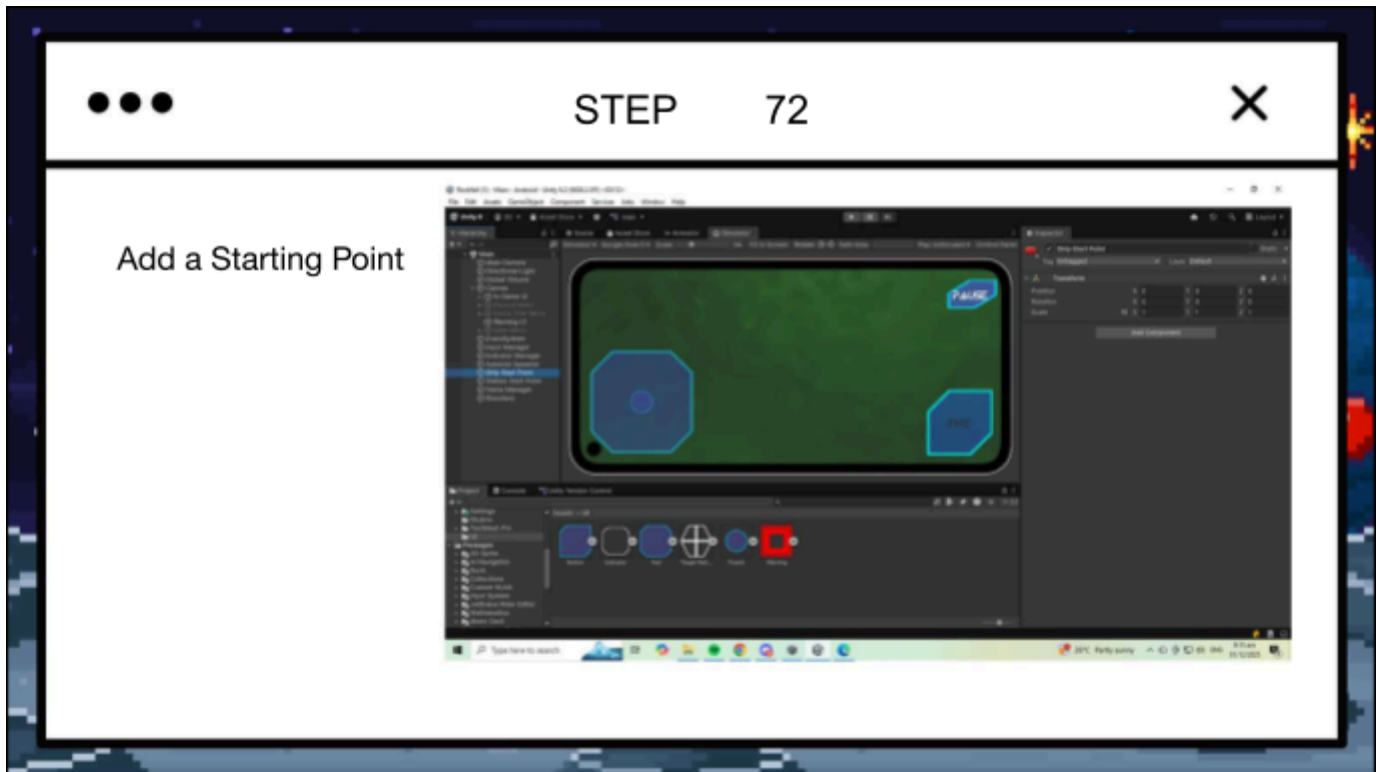
70



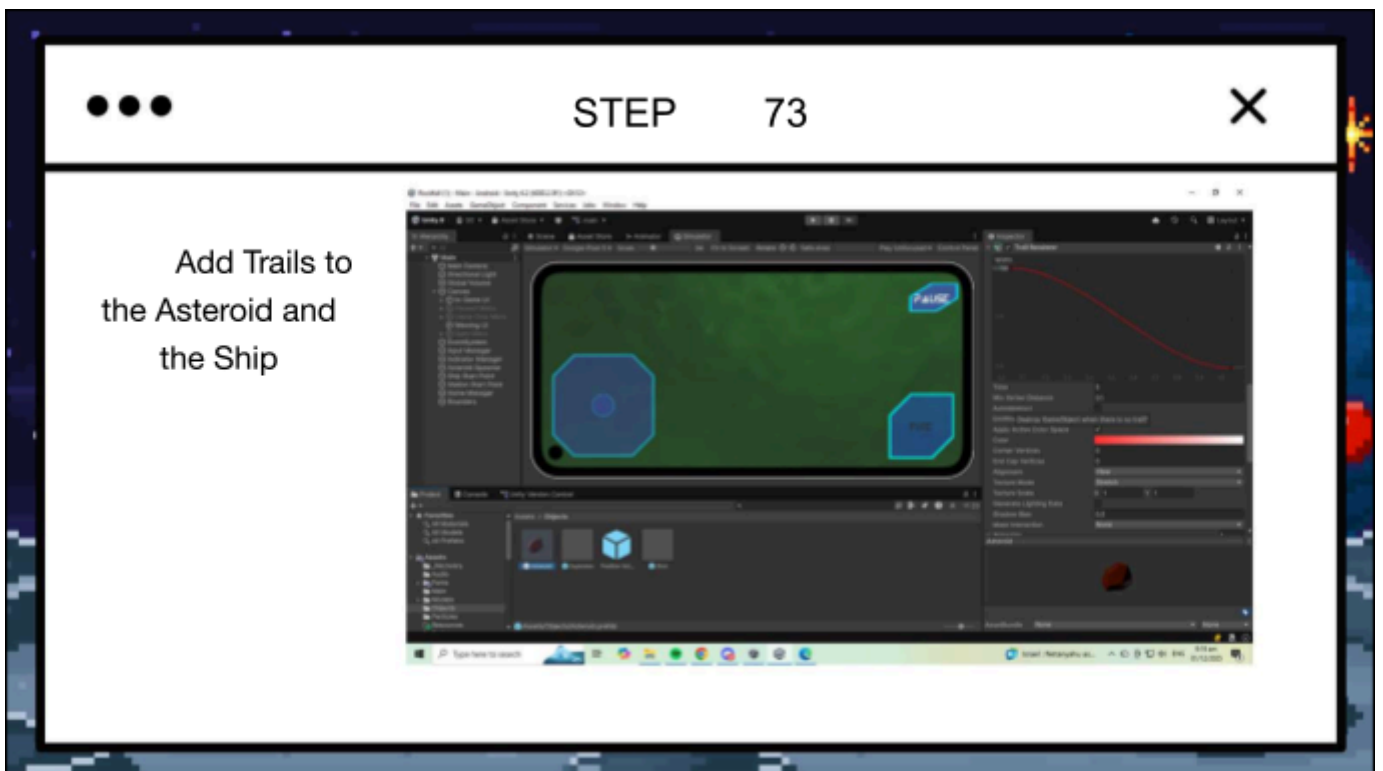
71



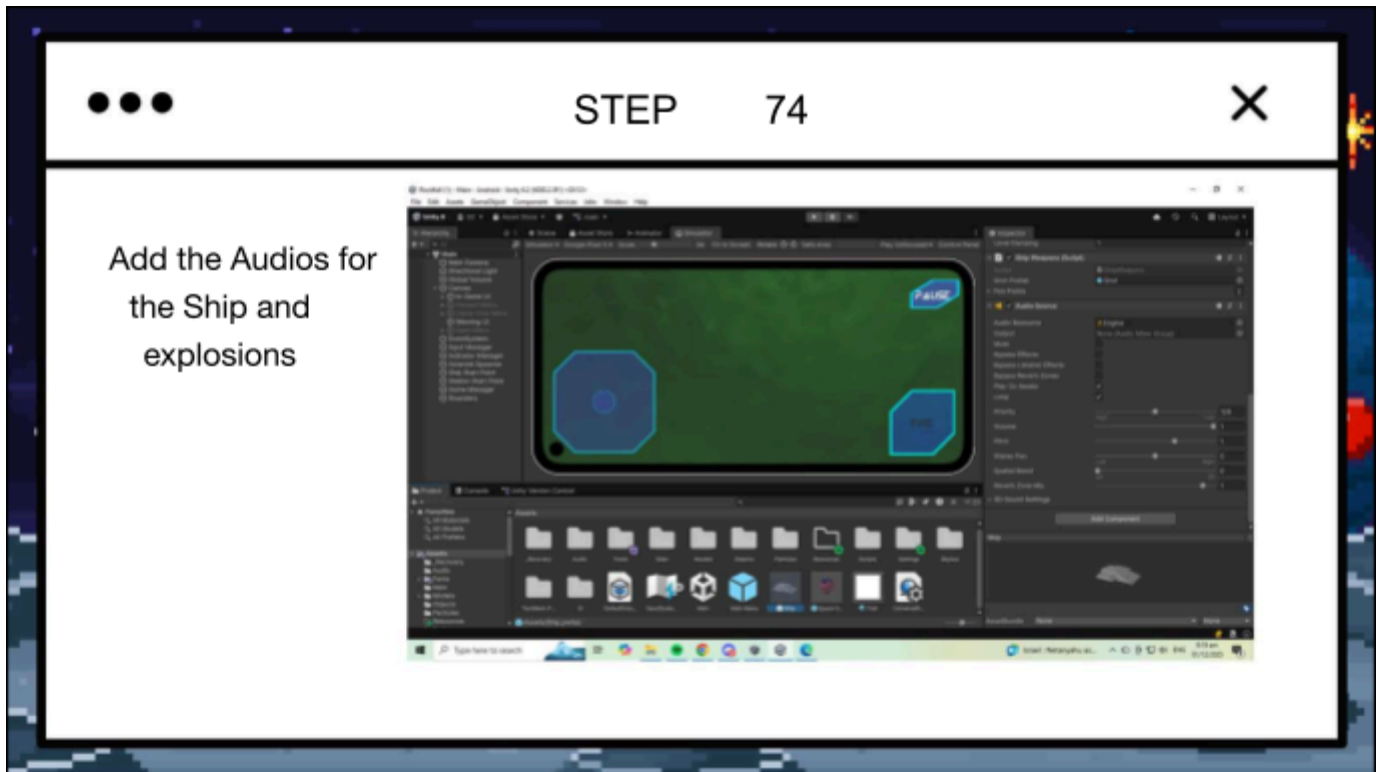
72



73



74



75



76